Machine Learning and Deep Learning
Lecture-05

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Overview

- Decision Trees
- Random Forests
- Gradient Boosting
- Dimensionality Reduction
- Principal Component Analysis (PCA)
- Singular value decomposition (SVD) [Not in Lecture Plan]

Decision Tree

Decision Trees (DT)

- Decision Trees can perform both classification and regression* tasks, including multi-output prediction.
  - Classification and Regression Trees (CART) are a non-parametric DT.
- DTs are used for inductive inference and for approximating discrete- and continuous-valued target functions.
- DTs are robust to noisy data, can learn disjunctive expressions, and are capable of fitting complex datasets.
- DTs represent decisions through a sequence of if–else rules, leading to an interpretable decision process.
- Split Point: A split point occurs at each internal node of the tree where a decision is made.
- Suffers from overfitting.

* estimate the relationships between a dependent variable ('outcome variable') and one or more independent variables.

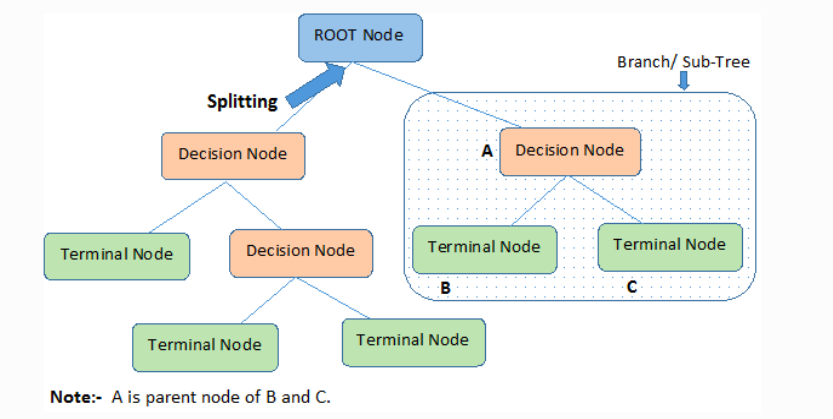
Decision Trees

- A Decision Tree searches a fully expressive hypothesis space.
- It can represent any discrete-valued function defined over finite attributes.
  - If the data has a pattern, a deep enough tree can theoretically capture it.
- A Decision Tree is essentially a set of logical rules. Formally, it represents a disjunction of conjunctions (OR of ANDs) of constraints on attribute values.
  - Conjunction (AND): Each individual path from the root to a leaf is a conjunction.
    - Example: (Outlook = Sunny) AND (Humidity = High) -> No.
  - Disjunction (OR): The entire tree is a disjunction of these paths.
    - Example: (Path A) OR (Path B) OR (Path C) -> Not Going out.
- Learned function is stored as a hierarchical tree structure, making it highly interpretable.
- To classify a new instance, you simply follow the path of attribute tests that match the instance's values until you reach a leaf node.

When to Consider Decision Trees?

- Use Decision Trees when you need to explain **why** a decision was made.
  - Instances are represented by attribute–value pairs.
  - Handles both categorical (discrete) and numerical values (without needing extensive preprocessing).
  - Best suited for classification tasks (Boolean or Multi-class)
- Essential when the target concept is "fragmented" meaning there are many different, unrelated ways to reach the same result.
  - Generally robust to errors or "noise" in the training data
  - Can handle missing attributes.
- In linear models, we use lambda to control complexity
- Use **Max Depth** to keep that "fully expressive" nature from capturing too much noise.

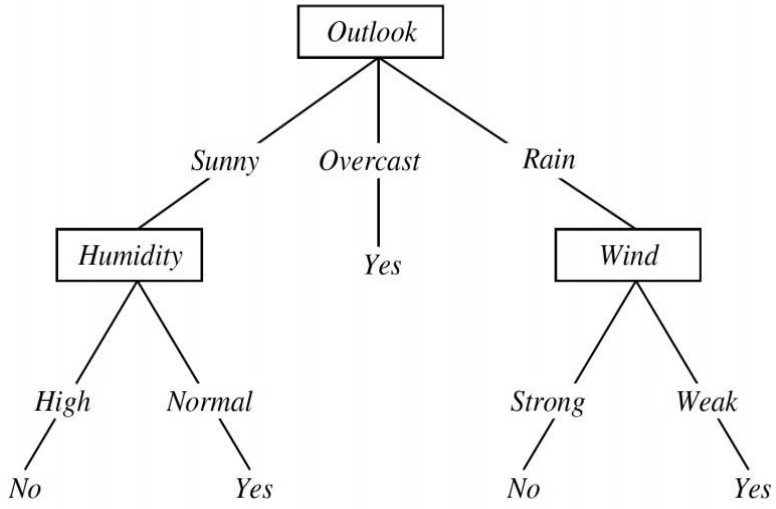
Decision Tree Flow



1. You start at the top (**Root**).
2. You answer a question (**Node**).
3. You follow the path that matches your answer (**Branch**).
4. You repeat until you reach a conclusion (**Leaf**).

Example: To classify a "Sunny" day with "High" humidity:

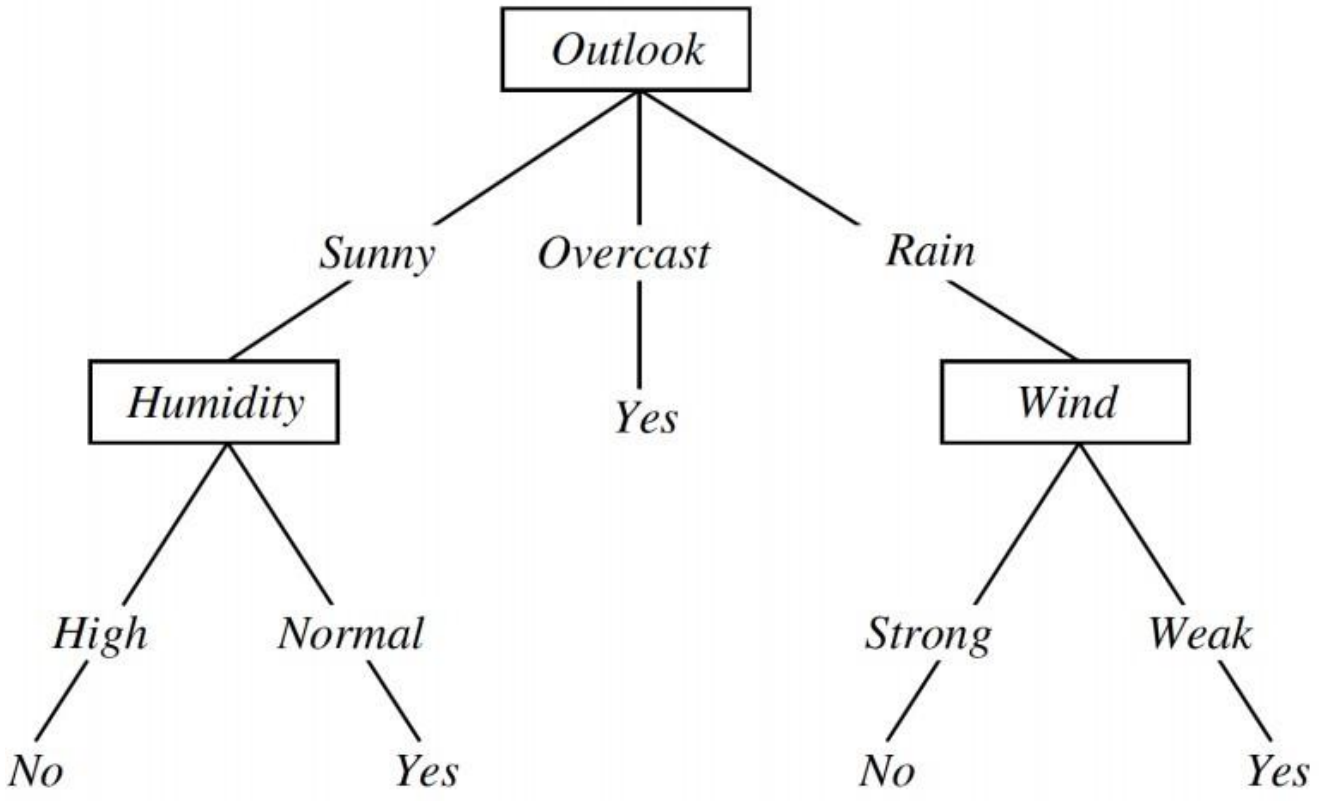
- Root: Outlook? → Sunny
- Internal Node: Humidity? → High
- Leaf: Result → Not Going Out



Example

(Outlook = Sunny ∧ Humidity = Normal) v (Outlook = Overcast) v (Outlook = Rain ∧ Wind = Weak)

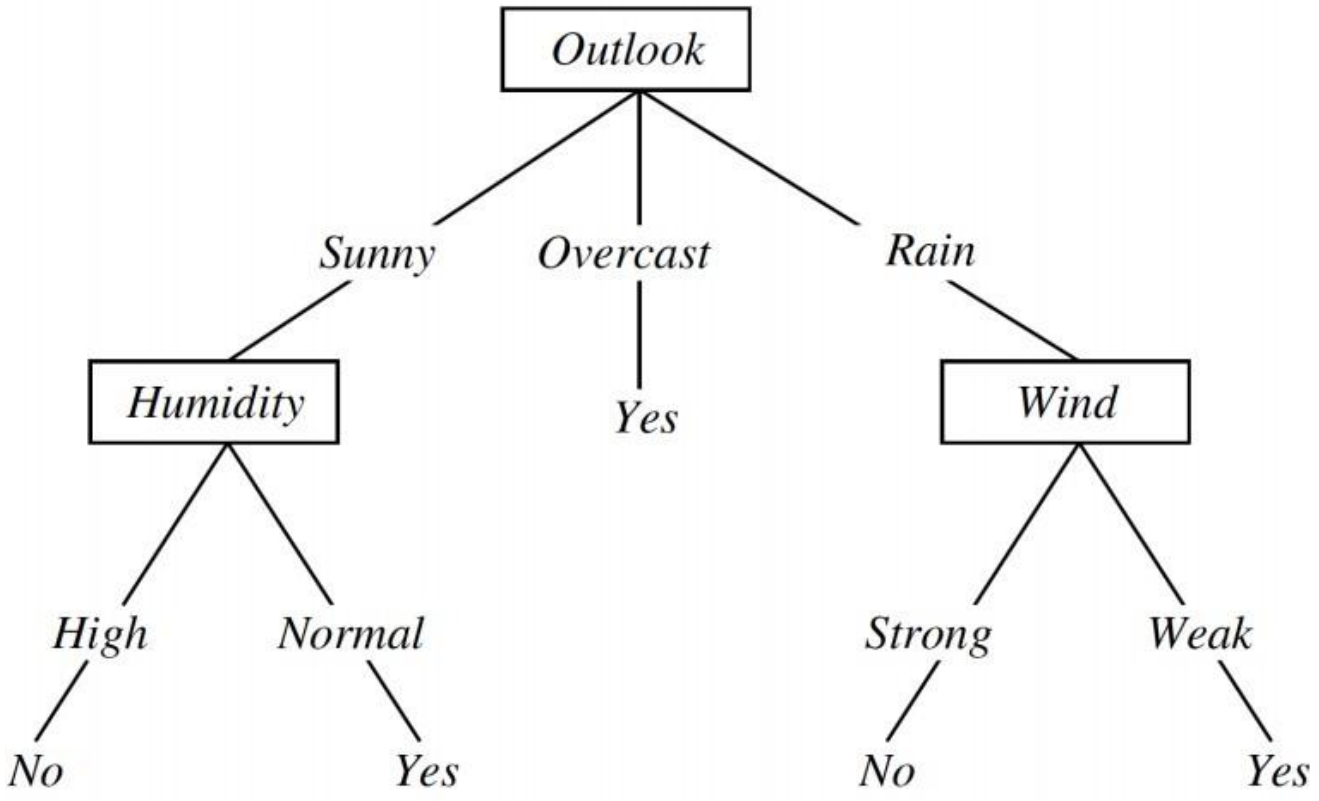
(Outlook=Sunny, Humidity=High, Wind=Strong) ==> negative instance.



Example

(Outlook = Sunny ∧ Humidity = Normal) v (Outlook = Overcast) v (Outlook = Rain ∧ Wind = Weak)

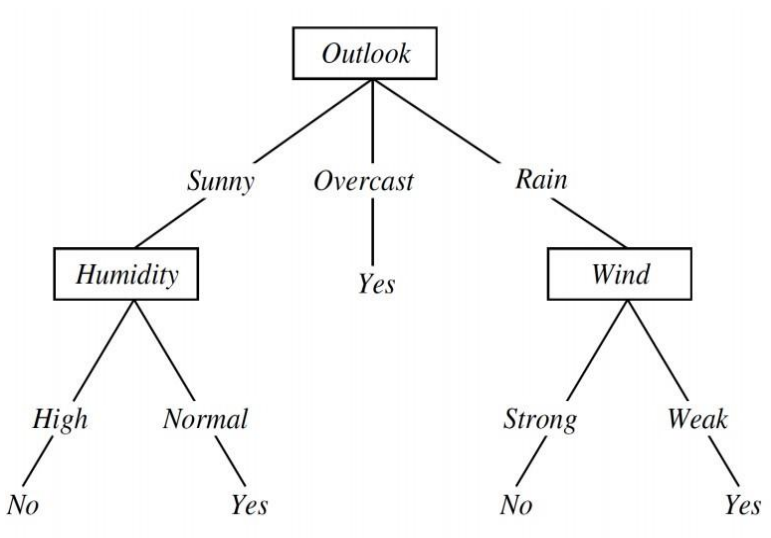
(Outlook=Sunny, Humidity=High, Wind=Strong) ==> negative instance.

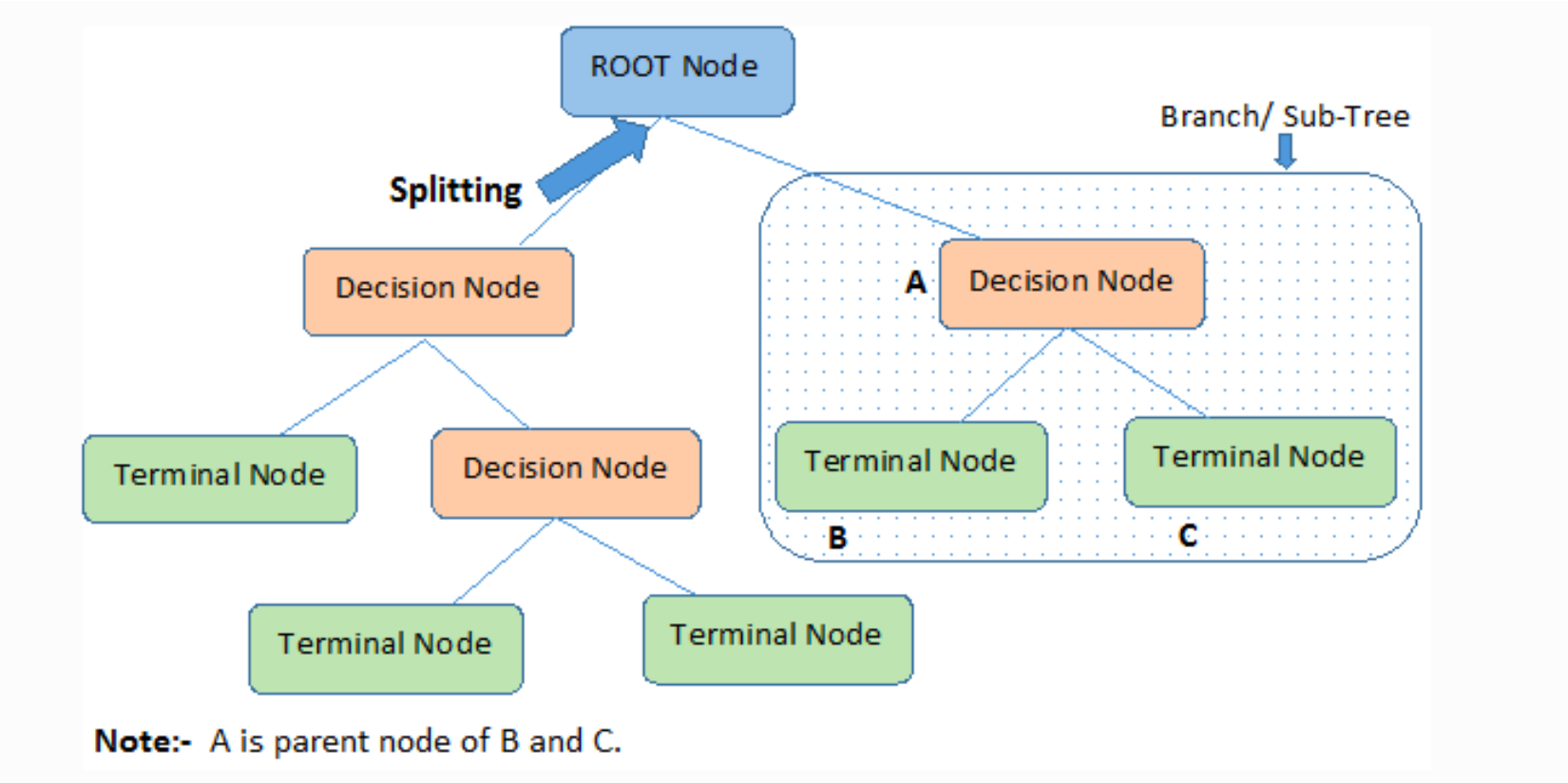


Example

(Outlook = Sunny ∧ Humidity = Normal) v (Outlook = Overcast) v (Outlook = Rain ∧ Wind = Weak)

(Outlook=Sunny, Humidity=High, Wind=Strong) ==> negative instance.

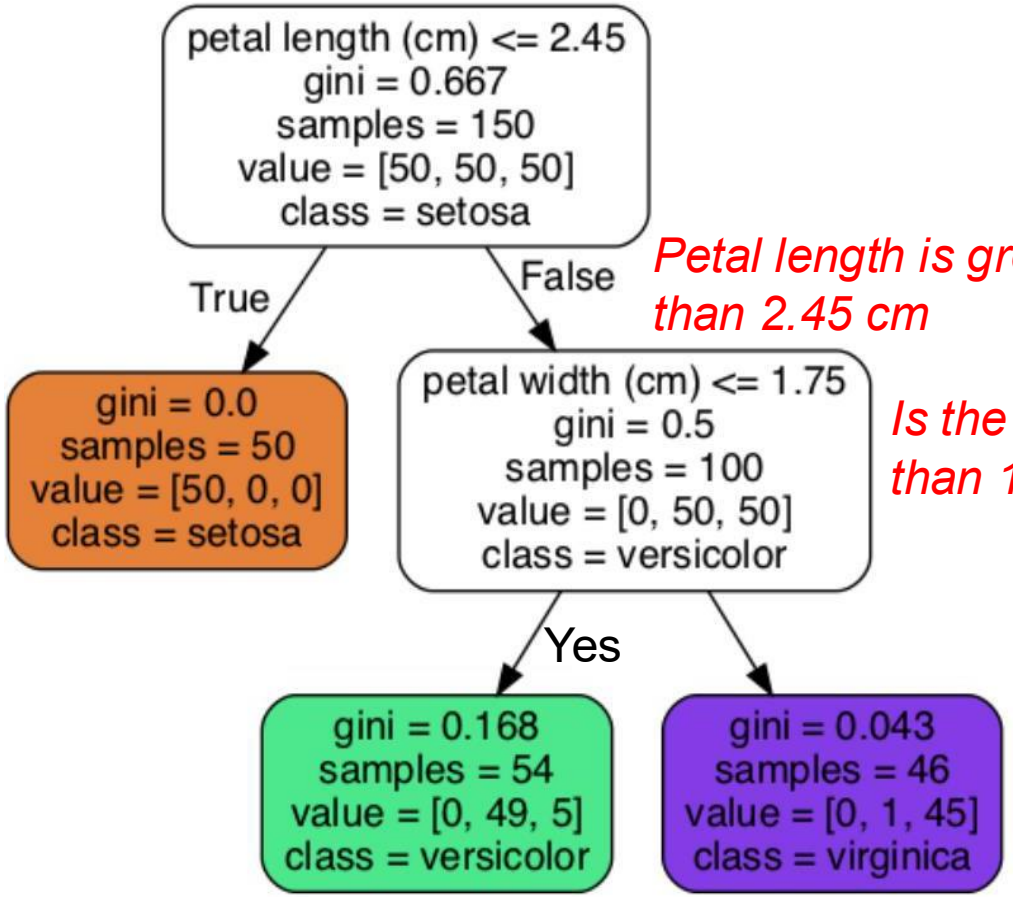




Source: Kdnuggets

Training and Visualizing a Decision Tree

Q. You want to find classify an iris flower



Petal length is greater than 2.45 cm

Is the petal width smaller than 1.75 cm?

- Require very little data preparation.
- Don't require feature scaling.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron. 2019.

Learning Decision Trees

- Given a set of training data in the form of examples (e.g., labeled instances), the goal is to construct questions that can be asked about the data.
- Learning a DT involves searching for the “best” tree to describe the training data.
- In principle, one could enumerate all possible trees and evaluate each one.
- However, even with a small number of features (e.g., three), the number of possible trees is extremely large, making exhaustive search computationally infeasible.
- DTs are constructed greedily, by selecting the most informative question at each step.
- Each internal node represents a question that splits the data.
- Learning a DT thus amounts to choosing the appropriate tests for the internal nodes.

Decision Tree Training Algorithm

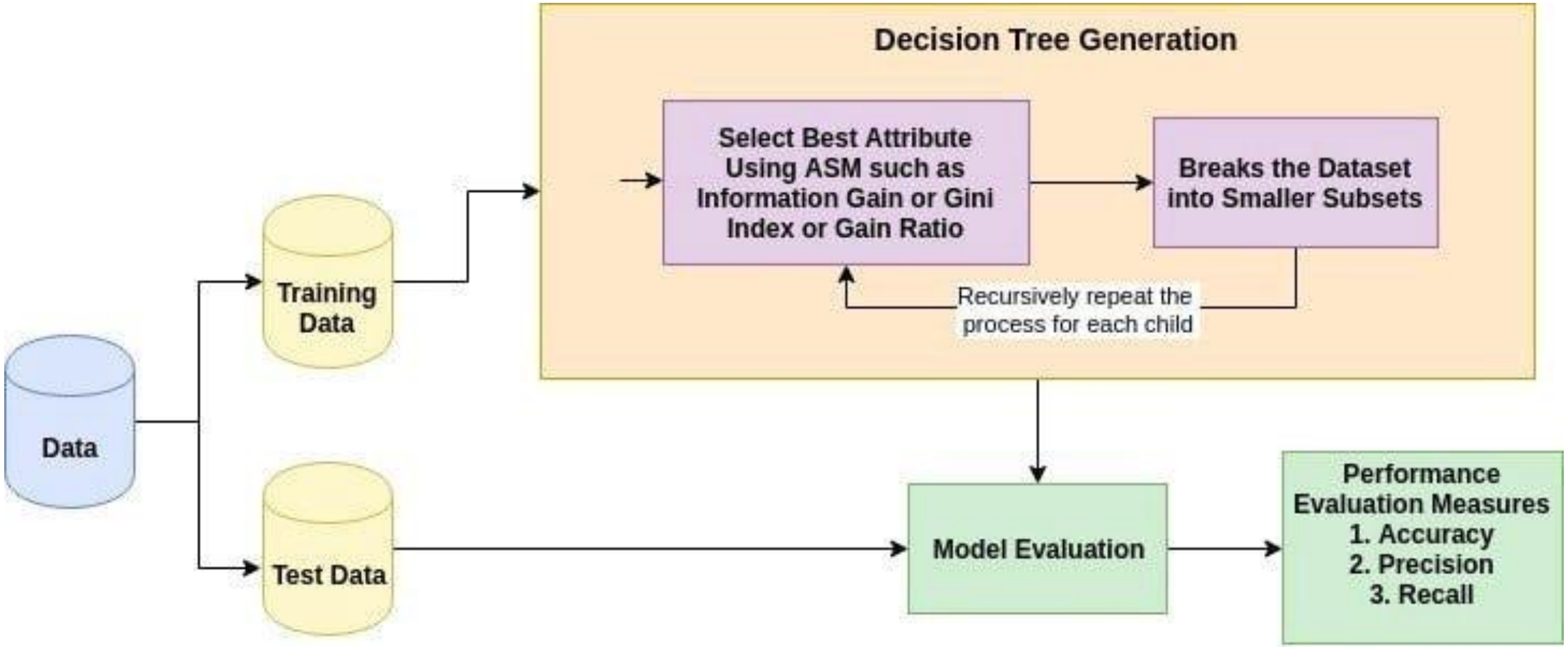
1. Start at the Root: Consider all available attributes.
2. Pick the "Best" Attribute: Choose the one that most effectively splits the data into pure classes (using Information Gain).
3. Recursive Splitting: Create branches for each value of that attribute and repeat the process for each branch.
4. Stop: When all instances in a branch belong to the same class or no attributes remain.

Selection Metric

1. Entropy: Measures the "impurity" or randomness of the data.
2. Information Gain: The reduction in entropy achieved by splitting on a specific attribute. The goal is to maximize this gain.

Decision Trees

ASM: Attribute Selection Measure



Source: DataCamp

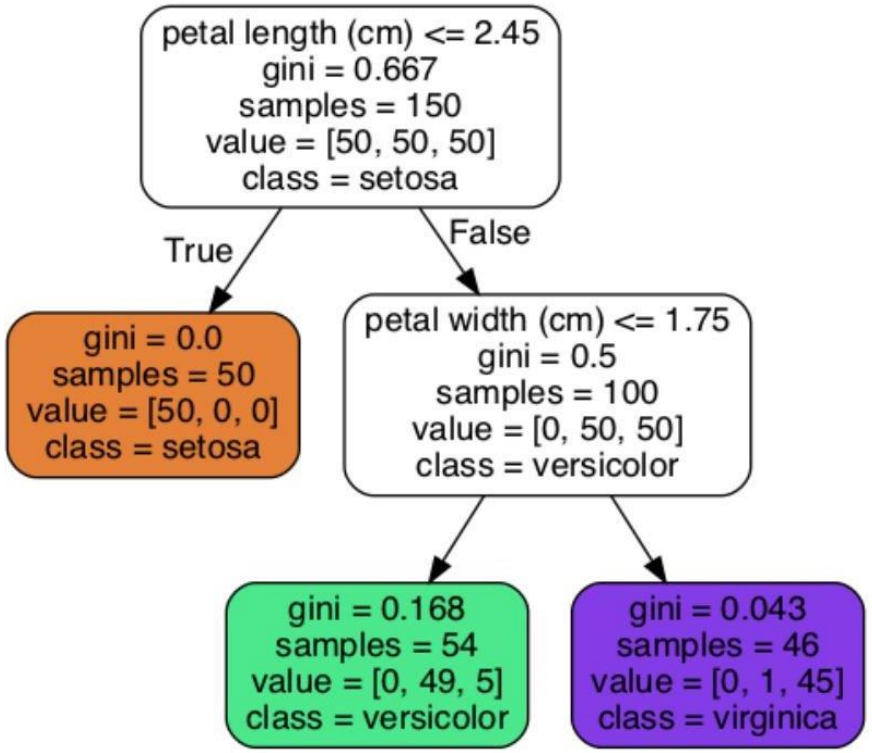
Impurity: Gini and entropy

- Gini Index (or Gini Impurity) measures the probability that a randomly chosen instance from a node would be misclassified (if it were labeled according to the specific class distribution of that node).
- It ranges from 0 (Perfectly Pure) to 0.5 (Max Impurity for binary classes, data is split 50/50).
- Calculates for all possible attribute splits and selects the split that results in the **lowest weighted impurity** for the child nodes.
- Primarily used for classification tasks with categorical targets (both binary and multi-class).
- Gini helps find the "clearest" threshold to separate the two groups.

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

$p_{i,k}$: ratio of instances of class k in node i

For multiclass problems, the maximum Gini impurity can be greater than 0.5. (classes are perfectly balanced)



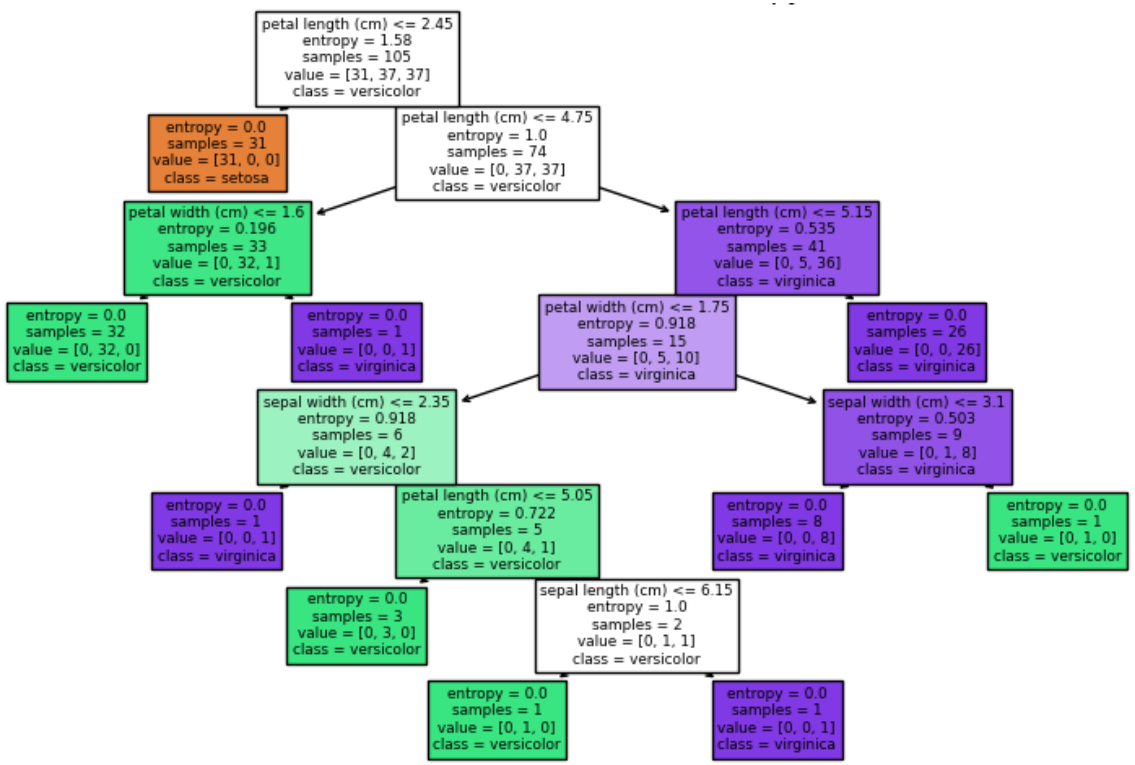
Gini impurity at the root node can be more than 0.5.

- It means the root node is evenly split among the different classes (see the value = [50, 50, 50]), making it harder to classify the data points correctly without further splitting that's what we did.

Impurity: Gini and Entropy

- Entropy measures the degree of disorder/uncertainty in a dataset.
  - Entropy = 0 (all instances belong to a single class).
  - Entropy = 1 (set is at maximum disorder (in a binary case, a perfect 50/50 split)).
- It is the difference between the Parent Node's Entropy and the Weighted Average Entropy of the Child Nodes.
  - Calculates the Information Gain for every possible attribute.
  - Selects the attribute that provides the highest Information Gain

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2 \left(p_{i,k}\right)$$



Gini and Entropy result in the same tree structure, but Entropy is slightly more computationally expensive due to the log calculation.

Range: 0 (perfectly pure) to log2(n) (maximum impurity, n= # of class)

Ensembles

Ensemble Learning

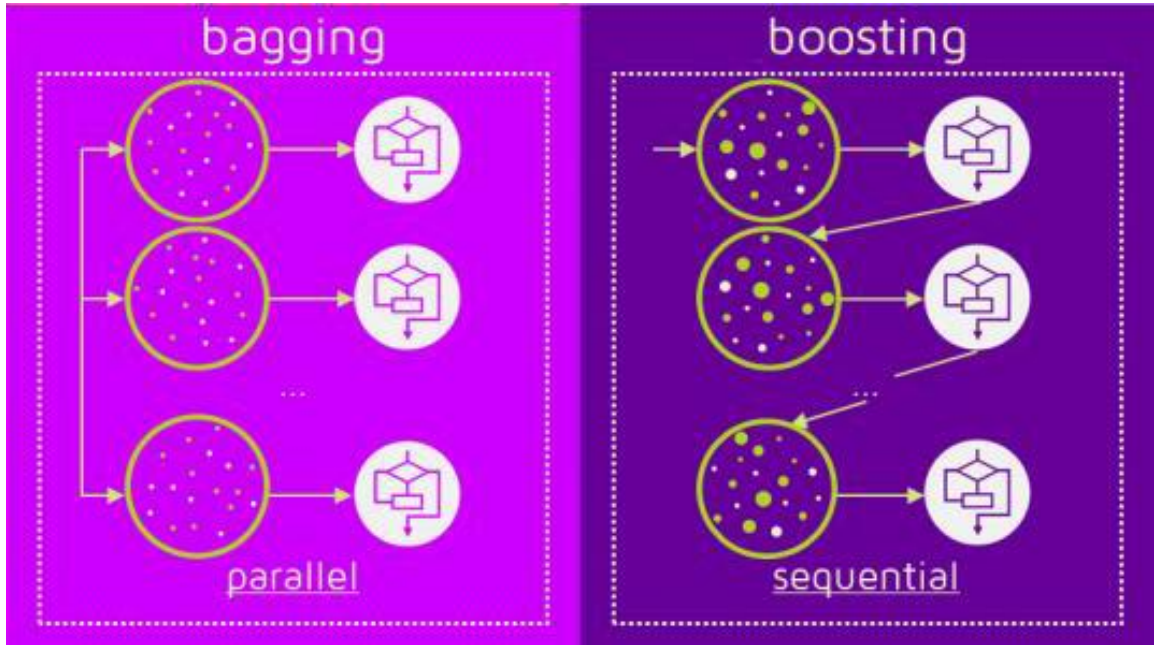
- Noise, variance, and bias are important factors in machine learning based prediction.
- Ensemble methods can reduce variance and, in some cases, bias.
- An ensemble is a collection of predictors whose outputs are combined to produce a final prediction.
- Commonly classified into Bagging and Boosting approaches.
- Aggregating several predictors is often more effective than using a single predictor.
- Multiple “weak” learners can be combined to form a “strong” learner.
- Differences among ensemble learning approaches include:
  - Which models are combined?
  - How are their predictions combined?

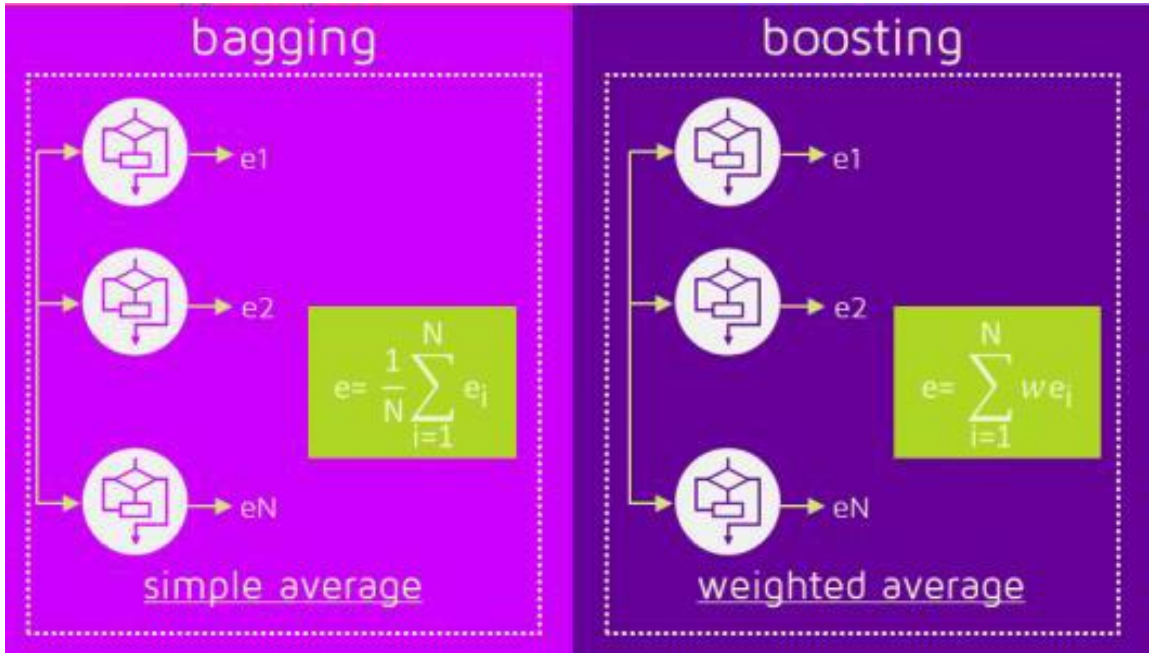
Bagging and Boosting

- **Bagging** builds many independent predictors and combines them using model-averaging techniques, such as majority voting (for classification) or averaging (for regression).
- Random Forests are a prominent example of a bagging-based ensemble method.
- It helps to reduce overfitting by lowering model variance.

- **In Boosting**, predictors are built sequentially, with each new model focusing on the errors made by previous models.
- Gradient Boosting is a well-known boosting technique.
- Boosting methods can overfit if not properly regularized.
- Ensemble methods generally work best when the individual predictors are as diverse (uncorrelated) as possible.

Bagging and Boosting





- Train classifiers using different algorithms.
- Will make different types of errors, improving ensemble's accuracy.

Img src: https://quantdare.com/what-is-the-difference-between-bagging-and-boosting/

Random Forests

- A Random Forest is a collection (ensemble) of multiple "constrained" Decision Trees.
- Combines individual tree predictions via Majority Voting to produce a single, more accurate, and robust classification.
  - While individual trees may overfit, their collective average cancels out errors.

- RF uses two key techniques:
  - Bagging: Each tree is trained on a random sample of the data, drawn with replacement.
  - Feature Randomness: At every split in every tree, the algorithm only considers a random subset of features. This prevents one "dominant" feature from making all trees look exactly the same.

- Standard Decision Trees are prone to overfitting.
- RF offers higher stability and handling "noisy" datasets better than a single tree.
  - 100–500 trees is often the "sweet spot."
  - Adding more trees to a Random Forest will not cause overfitting.

Random Forests

```
from sklearn.ensemble  import RandomForestClassifier

rnd_clf = RandomForestClassifier(n_estimators=500,
max_leaf_nodes=16,  n_jobs=-1) rnd_clf.fit(X_train,  y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

- Lower number of leaf nodes restricts the depth of each tree -> reducing overfitting
- Many trees → lower variance, stable predictions
- Shallow trees (max_leaf_nodes=16) → lower overfitting
- n_jobs=-1: Uses all available CPU cores
- Each tree is trained on a bootstrap sample of X_train.
- At each split, a random subset of features is considered.
- y_train contains the class labels.

```
        Training Data (X_train, y_train)
                      |
                      |
      -----------------------------------
      |               |               |
Bootstrap Sample 1  Bootstrap Sample 2  Bootstrap Sample 3   ... (500 samples)
      |               |               |
   Decision Tree 1   Decision Tree 2   Decision Tree 3        ... (500 trees)
  (max 16 leaves)  (max 16 leaves)  (max 16 leaves)
      |               |               |
      -----------------------------------
                      |
               Majority Voting
                      |
             Final Class Prediction
```

Random Forests: max_features

- In Scikit-Learn, the number of random features to consider at each split is controlled by max_features.
- For Classification, the default is usually m = sqrt(total features).
- For Regression, it is usually m = (total features)/3.

Random Forests: Feature importance

- Feature importance measures how useful each feature is for making predictions.
- Feature importance is computed based on the reduction in impurity (e.g., Gini) contributed by a feature across all trees and splits.
- Each time a feature is used to split a node, the impurity decreases.
- Features that:
  - are used more frequently across the forest, and
  - produce larger impurity reductions, are considered more important.

- Final importance score is a weighted average of impurity reductions, averaged across all trees in the ensemble.

- During tree construction, only a random subset of features is considered at each split.
- Some variants further increase randomness by using random split thresholds.

Random Forests

- Ex: Trains a RandomForestClassifier on the iris dataset and outputs each feature's importance.

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()

rnd_clf = RandomForestClassifier(n_estimators=500, n_jobs=-1)
rnd_clf.fit(iris["data"], iris["target"])

for name, score in zip(iris["feature_names"], rnd_clf.feature_importances_):
    print(name, score)
```

sepal length (cm) 0.112492250999
sepal width (cm) 0.0231192882825
petal length (cm) 0.441030464364
petal width (cm) 0.423357996355

Most important features are petal length (44%) and petal width (42%)

- Computes Gini-based feature importance.
  - iris["data"] → feature matrix (150 × 4)
  - iris["target"] → class labels (3 flower species)
  - iris["feature_names"] → feature names
  - feature_importances_: Array of length 4 (one per feature)
    - Each value represents the relative importance of a feature
    - Values sum to 1
- zip(...) pairs: Feature names with their importance scores

Boosting

- In Boosting, each subsequent predictor learns from the errors of previous predictors.
- Training observations are assigned unequal weights, so misclassified or poorly predicted instances receive greater emphasis in subsequent models.
- Weak learners can be chosen from decision trees, regressors, or classifiers, depending on the task.
- Gradient Boosting is a popular boosting technique that builds models sequentially by minimizing a loss function.
- Gradient Boosting often slower than Random Forests.
- Careful stopping criteria (e.g., learning rate, number of trees, tree depth, early stopping) are essential to prevent overfitting.

Gradient Boosting

- Gradient Boosting combines boosting with gradient descent in function space to minimize a loss function.
- Gradient Boosting works by sequentially adding predictors to an ensemble, where each new predictor corrects the errors of the previous ones.
- At each stage, the new predictor is trained to fit the residual errors (or, more generally, the negative gradient of the loss function) produced by the current ensemble.
- The final prediction is obtained by summing the contributions of all predictors in the ensemble.
- Variants include:
  - AdaBoost (Adaptive Boosting)
  - XGBoost (Extreme Gradient Boosting)

Gradient Boosting

- Steps in Gradient Boosting:
  - Fit an additive model (ensemble) in a forward stage-wise manner.
  - At each stage, introduce a weak learner to compensate for the shortcomings of the existing ensemble.
  - In Gradient Boosting, these “shortcomings” are identified using the gradients of the loss function.
  - The gradients indicate how the model should be updated to improve performance.

Learning rate shrinks the contribution of each tree by learning_rate

```
from sklearn.ensemble import GradientBoostingClassifier
GradientBoostingClassifier(n_estimators=100, learning_rate=1.0, ... max_depth=1,
random_state=0).fit(X_train, y_train)
```

Maximum depth of the individual regression estimators. (limits the no of nodes in tree)

No of boosting stages to perform.

Gradient Boosting

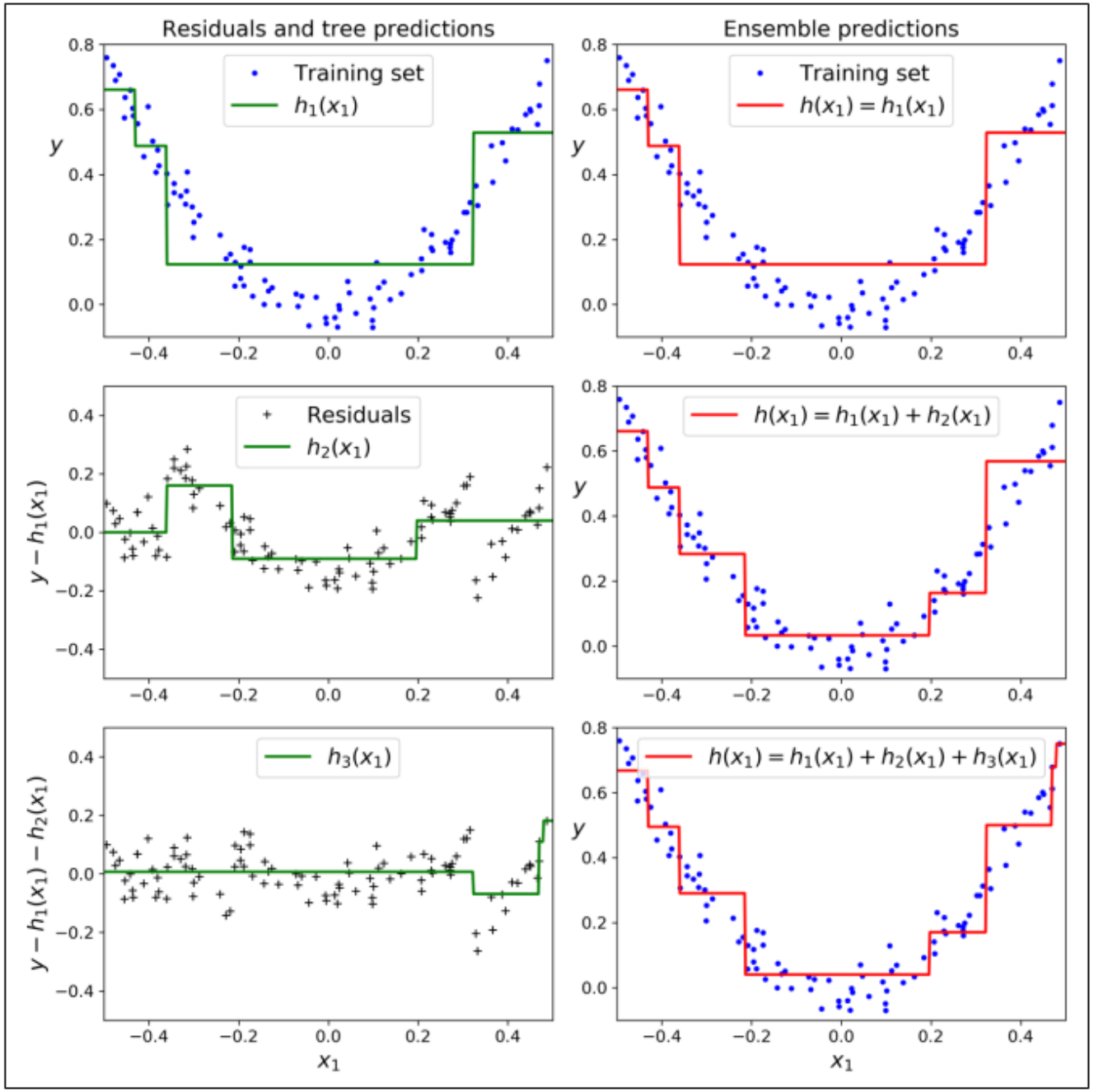
- Step 1: Ensemble begins with a single base learner. The ensemble's predictions are identical to the first tree's output.
- Step 2 Learning from Residuals:
  - A second tree is trained on the residual errors (the difference between the actual value and Tree 1's prediction).
  - Ensemble's new prediction is the sum of the first tree and the corrected second tree.
- Step 3: Sequential Process: This process repeats.
  - A third tree is trained on the residuals of the second tree's contribution, and so on.
  - With each new tree added to the ensemble, the "gap" between the prediction and the true value shrinks.
- Step 4. Convergence:
  - Ensemble's predictions get progressively better as more trees are added.
  - Focusing specifically on the data points that were difficult for previous trees to predict, the model achieves very high accuracy.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron. 2019.

Gradient Boosting

1. Ensemble has one tree, its predictions are same as first tree's predictions.
2. A new tree is trained on residual errors of first tree. On right see that ensemble's predictions are equal to sum of predictions of first two trees.
3. Another tree is trained on residual errors of second tree.

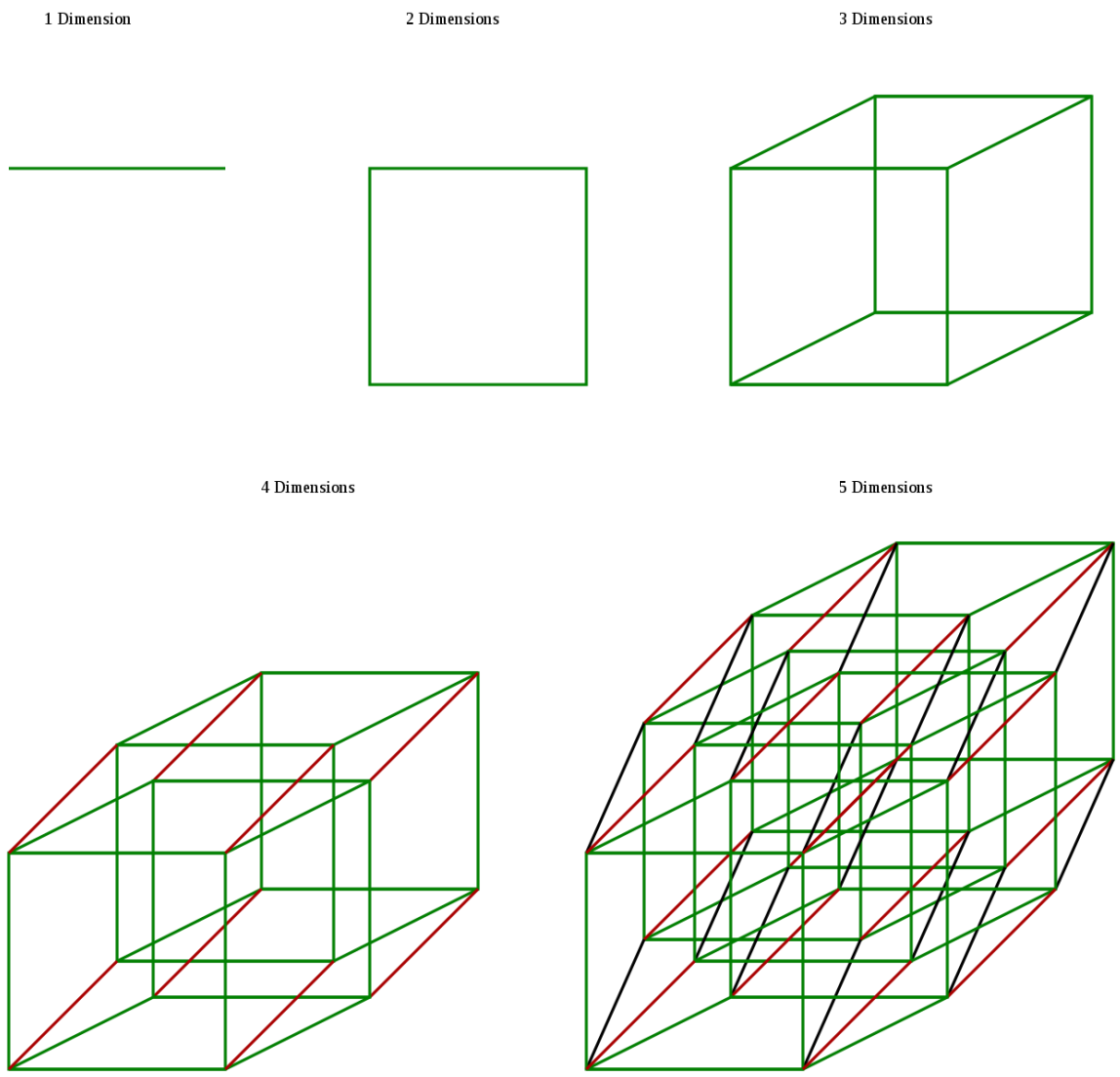
- Ensemble's predictions gets better as trees are added to ensemble.



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron. 2019.

Dimensionality Reduction

Dimension



- Dimension of an object can be defined as minimum number of coordinates needed to specify any point within it.

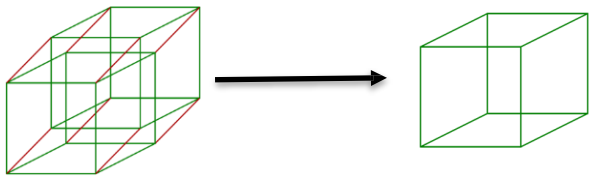
Src: Wikipedia

Dimensionality Reduction

- Process of deriving a set of degrees of freedom which can be used to reproduce most of the variability of a data set.

- Goal: To reduce dimensions by removing redundant and dependent features.

- How? By transforming features from higher dimensional space to a lower dimensional space.



- Approaches: Unsupervised and Supervised

Dimensionality Reduction

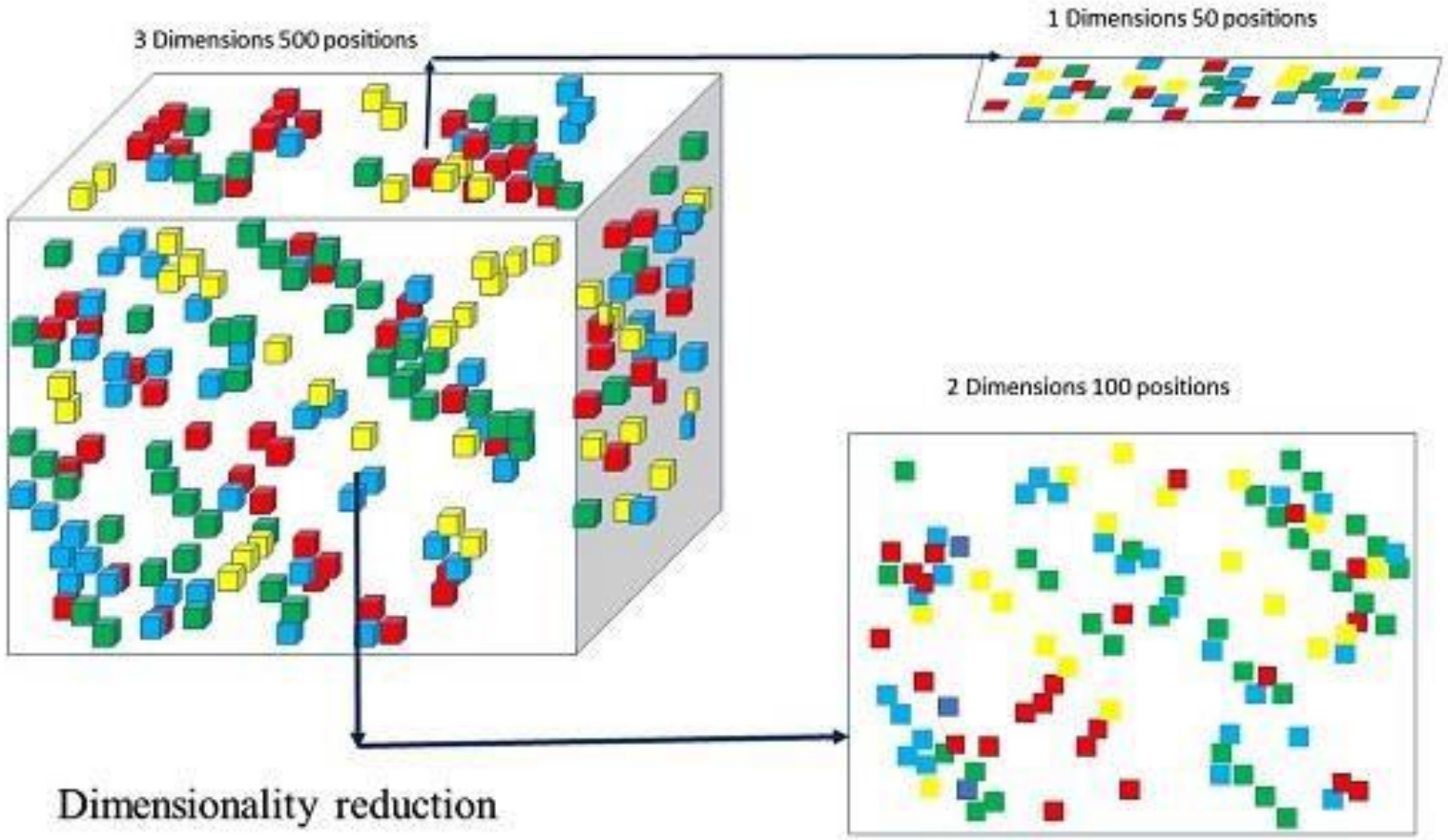
- Approaches: Unsupervised and Supervised
    - Unsupervised where no need for labeling classes of data.
        - Independent Component Analysis (ICA)
        - Non-negative Matrix Factorization (NMF)
        - Principal Component Analysis (PCA)
            - Ideal for visualization and noise removal.

    - Supervised where class labels are considered.
        - Mixture Discriminant Analysis (MDA)
        - Linear Discriminant Analysis (LDA)
            - Ideal for biometrics, Bioinformatics, and chemistry.

In a nutshell

	Supervised Learning	***Unsupervised Learning***
Discrete	classification or categorization	clustering
Continuous	regression	dimensionality reduction

Dimensionality Reduction



- Application:
  - Machine learning
  - Data mining
  - Bioinformatics
  - Biometric
  - Information Retrieval

Example of multidimensional cube. Perform slicing operations to separate dimensions.

Image Src: https://www.c-sharpcorner.com/article/a-quick-overview-of-machine-learning-tasks/

Principal Component Analysis (PCA)

The Camera Analogy:

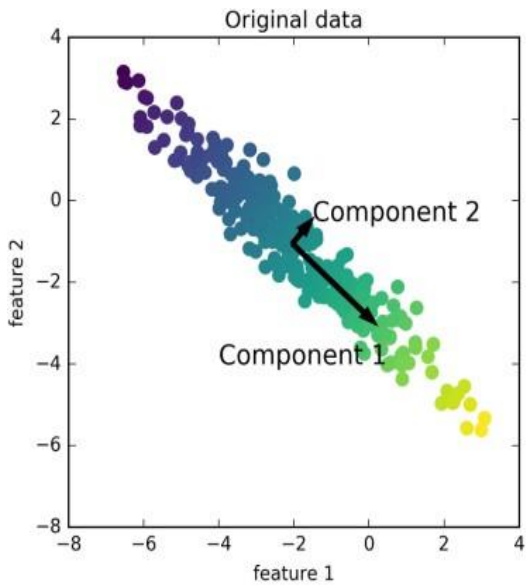
Imagine a 3D object, like a teapot. PCA is like finding the best angle to take a 2D photo of that teapot so that you can still tell exactly what it is. It finds the "best view" (the principal components) of your high-dimensional data.

Principal Component Analysis (PCA)

- PCA is a popular technique for dimensionality reduction.
- It is a classical linear method that characterizes linear subspaces in the data.
- PCA is applied to datasets consisting of observations on numerical variables.
- It is commonly used as an EDA tool.
- PCA is a simple, non-parametric method for extracting the most relevant information from high-dimensional datasets.

Principal Component Analysis (PCA)

- PCA reduces dimensionality by revealing the underlying structure in datasets.
- It is an unsupervised learning approach.
- PCA explains most of the variability in the data using a smaller number of variables (PCs).
  - It identifies orthogonal axes (principal components -> PCs) that account for the largest variance in the training set.
  - PCA applies a linear transformation (rotation) to the dataset to produce statistically uncorrelated features (PCs).
  - This transformation is followed by selecting a subset of the new features (PCs) based on their explained variance (importance).



Principal Component Analysis (PCA)

- PCA is influenced by the magnitude (scale) of each variable.
- Features are often standardized before applying PCA.
- PC1 of a dataset is a linear combination of the original features that captures the maximum possible variance.
- PC2 is a linear combination of the features that captures the maximum remaining variance, subject to being uncorrelated with PC1.
- All PCs are uncorrelated with each other.
- The PCs form an orthogonal basis of a new feature space, regardless of the original data dimensionality.

```
from sklearn.decomposition import PCA

pca = PCA(n_components = 2)
```

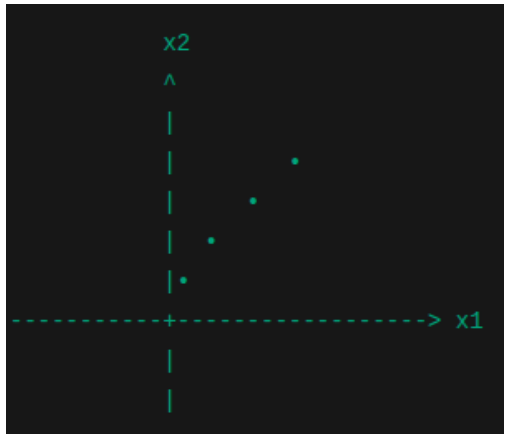
Principal Component Analysis (PCA)

- Primary steps in PCA calculation:
  1. Center (and usually standardize) the data.
  2. Compute the covariance matrix of the data.
  3. Compute the eigenvalues and eigenvectors of the covariance matrix.
  4. Sort the eigenvalues (and corresponding eigenvectors) in descending order.
  5. Select the top PCs based on the largest eigenvalues.

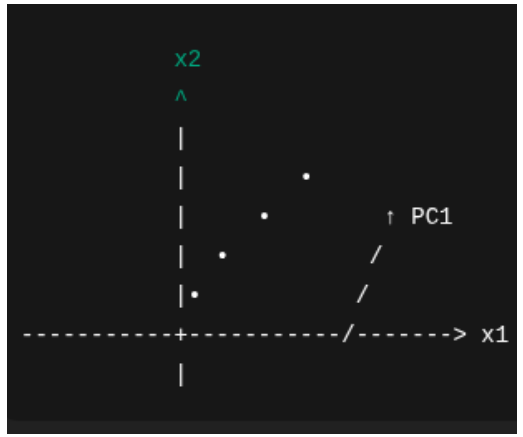
- Applications of PCA:
  1. Neuroscience: Used to analyze high-dimensional neural data (e.g., EEG) by identifying dominant patterns of neural activity.
  2. Computer Graphics: Applied for image compression, shape analysis, and dimensionality reduction in rendering and animation.
  3. Atmospheric Science: Used to analyze large-scale climate and weather data by extracting dominant spatial and temporal patterns.

Principal Components (PCs)

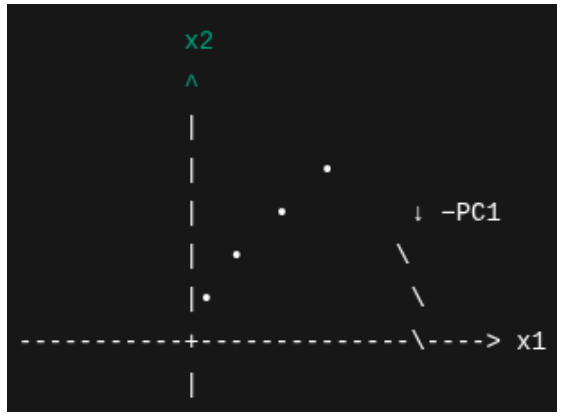
- Overall PCA calculation process: For each principal component (PC):
- PCA finds a zero-centered unit vector that points in the direction of the principal axis of maximum variance.
- The direction (sign) of PCA eigenvectors is not unique due to sign ambiguity, though the principal axes and subspace remain unchanged.
- If the training dataset is slightly perturbed and PCA is recomputed,
- the resulting unit vectors may point in the opposite direction compared to the original ones.
- Despite this sign change, the vectors still lie along the same principal axes and represent the same subspace.



Original dataset



PCA finds the principal axis (PC1) along this direction.



After slight perturbation of data

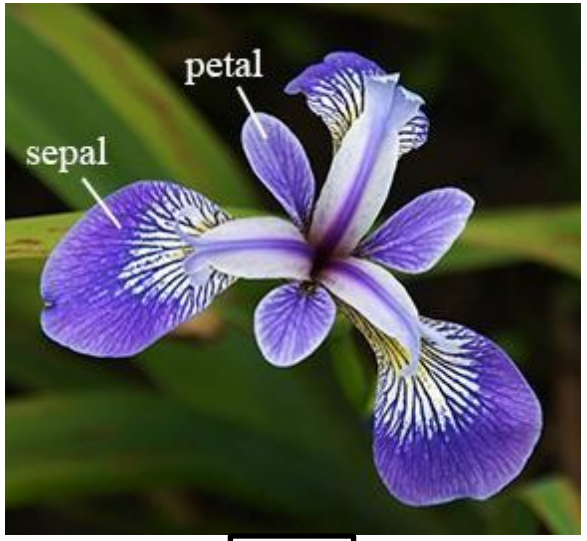
- PCA now returns −PC1 instead of PC1
- The vector points in the opposite direction
- Axis is identical

Principal Component Analysis (PCA)

- How do we know how many principal components (PCs) to use?
  - Proportion of Variance Explained (PVE) measures the percentage of total variance captured by each principal component.
  - The PVE of the *m-th* principal component is calculated as: (m-th eigenvalue) / (sum of all eigenvalues)
  - This indicates how much variance is retained by that component.
  - For a general data matrix X with **n** observations and **p** variables, there are at most $\min(n - 1, p)$ principal components.

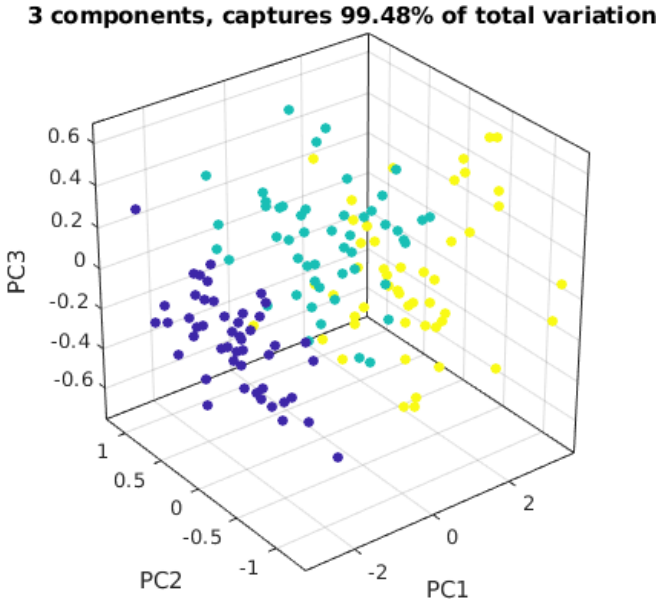
- There is no single fixed rule for determining how many PCs to retain.
- Choice depends on desired trade-off between dimensionality reduction and variance retention.

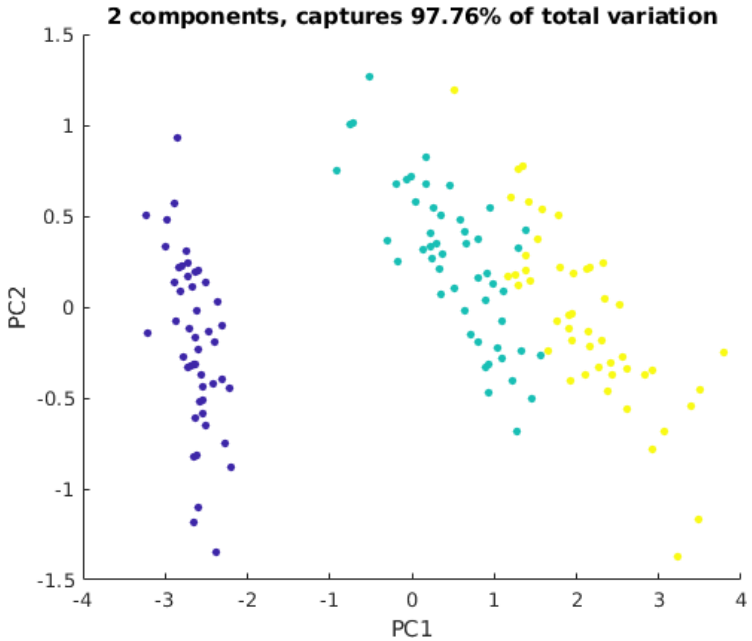
PCA Example: Iris data



- Total 150 iris flowers.
- Measurement: sepal length, sepal width, petal length, petal width

Flowers belong to three different species: blue, green, yellow

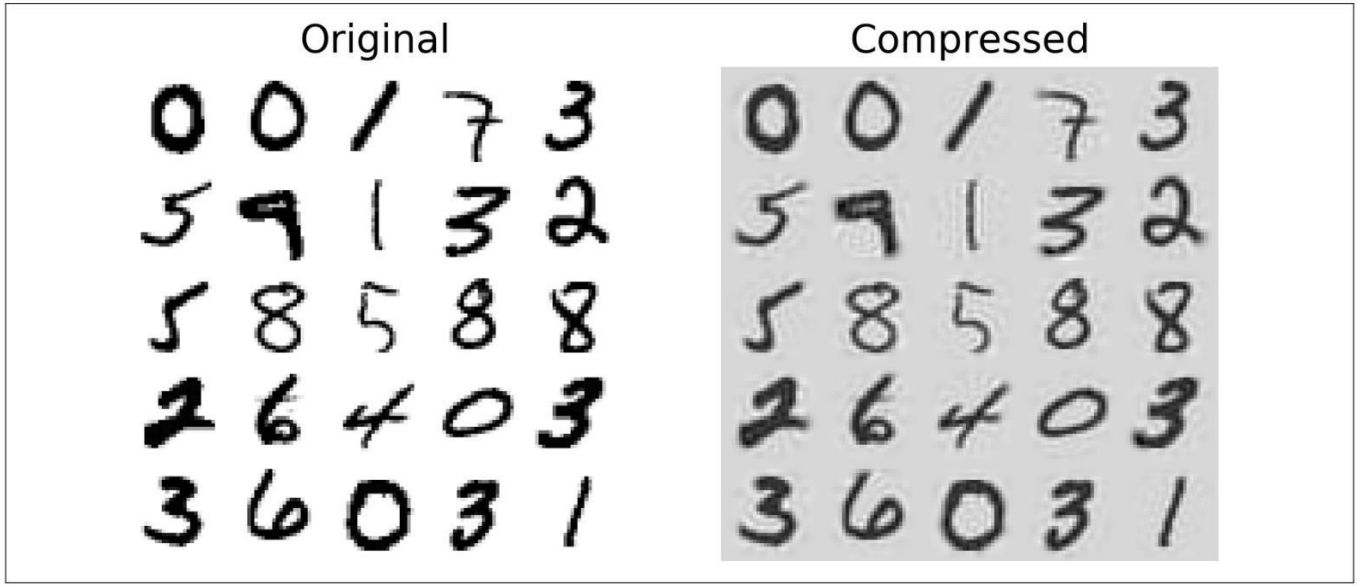




https://www.math.umd.edu/~petersd/666/html/iris_pca.html

PCA Example: MNIST Compression

- Apply PCA to MNIST dataset while preserving 95% of its variance.
- Each instance have just over 150 features, instead of the original 784 features.
- Dataset is now less than 20% of its original size.




```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

MNIST compression that preserves 95% of the variance

Géron, Aurélien. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. O'Reilly Media, 2019.

PCA Types

- Types of PCA:
- Randomized PCA efficiently computes an approximation of the first **d** principal components and is well suited for large, high-dimensional datasets.
  - Limitation: the entire dataset (or a large portion of it) typically needs to fit in memory.
- Incremental PCA (IPCA) splits the training data into mini-batches and updates the principal components incrementally, processing one mini-batch at a time.
  - Suitable for out-of-core or streaming data.
- Kernel PCA enables nonlinear dimensionality reduction by applying PCA in a high-dimensional feature space induced by a kernel function.

Incremental & Kernel PCA

- Incremental PCA (IPCA) finds a projection of the data that is similar to standard PCA by processing small batches of samples at a time, rather than the entire dataset at once.
- IPCA maintains and updates estimates of the principal components and explained variance incrementally as new data batches are processed.
- Kernel PCA can find a projection of the data that makes it more likely to be linearly separable in the transformed feature space.
- Kernel PCA supports forward transformation, but an exact inverse transform is not generally available and can only be approximated under certain conditions.

Singular Value Decomposition (SVD)

[Not in Lecture Plan]

Singular Value Decomposition (SVD)

- SVD is a method for
  - transforming correlated variables into a set of uncorrelated set
    - Exposes various relationships among original data items.
  - Identifying and ordering dimensions along which data points exhibit most variation.
    - Perform data reduction.
  - Power Method is used to compute SVD.

Singular Value Decomposition (SVD)

- Basic steps of SVD:
  - Consider a high dimensional, highly variable set of data points.
  - Reduce it to a lower dimensional space that exposes substructure of original data (more clearly).
  - Orders it from most variation to least.

Singular Value Decomposition (SVD)

- SVD decomposes any matrix (A) into three matrixes with special properties and whose multiplication gives back the same matrix: $A_{mn} = U_{mm}S_{mn}V^T_{nn}$
  - U: Orthogonal matrix
  - S: Diagonal matrix
  - V^T: Transpose of an orthogonal matrix

- Note: $U^TU = I$ (Identity Matrix), $V^TV = I$
  - Columns of U are orthonormal eigenvectors of AA^T
  - Columns of V are orthonormal eigenvectors of A^TA
  - S is a diagonal matrix containing square roots of eigenvalues from U or V in descending order.

Singular Value Decomposition (SVD)

- SVD can also be represened as: $A = \sum_{i=1}^{r} \sigma_i \mathbf{u_i} \mathbf{v_i}^T$
- Let A be a n×d matrix with singular vectors $v_1, v_2, \ldots, v_r$ and corresponding singular values $\sigma_1, \sigma_2, \ldots, \sigma_r$.
- Then $u_i = (1/\sigma_i)*Av_i$, for i = 1, 2, . . . , r, are left singular vectors. Where Rank(A) = r.
- A can be decomposed into a sum of rank one matrices as:
- $A = U\Sigma V^T$

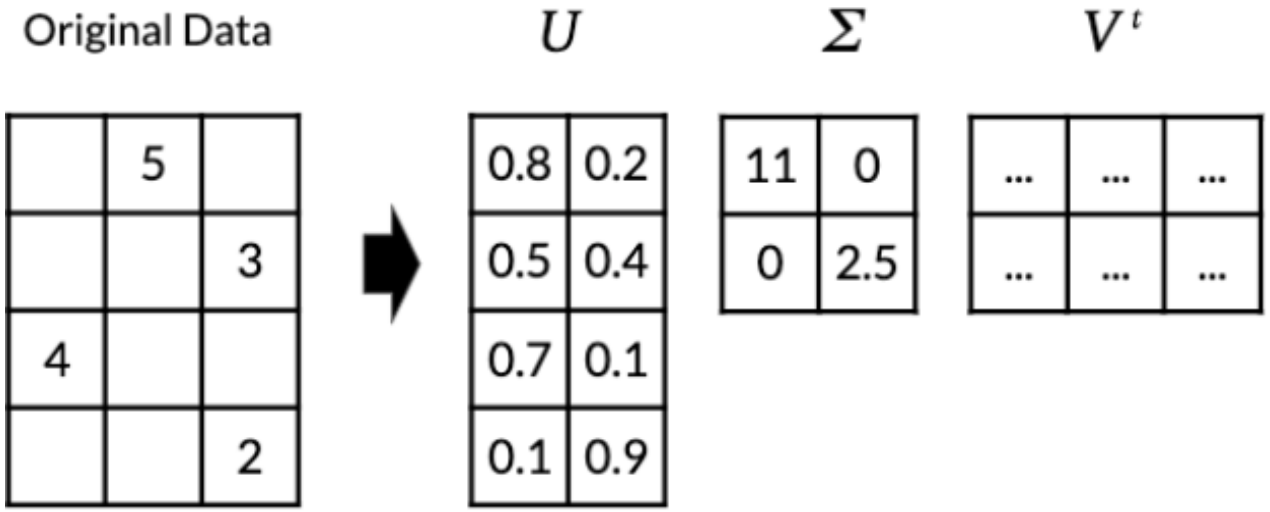
OR

$$A = \sum_{i=1}^{r} \sigma_i \mathbf{u_i} \mathbf{v_i}^T$$

$U \in R^{m \times r}$ has orthonormal columns; $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$, where $\sigma_1 \geq \cdots \geq \sigma_r > 0$; $V \in R^{n \times r}$ has orthonormal columns.

Singular Value Decomposition (SVD)

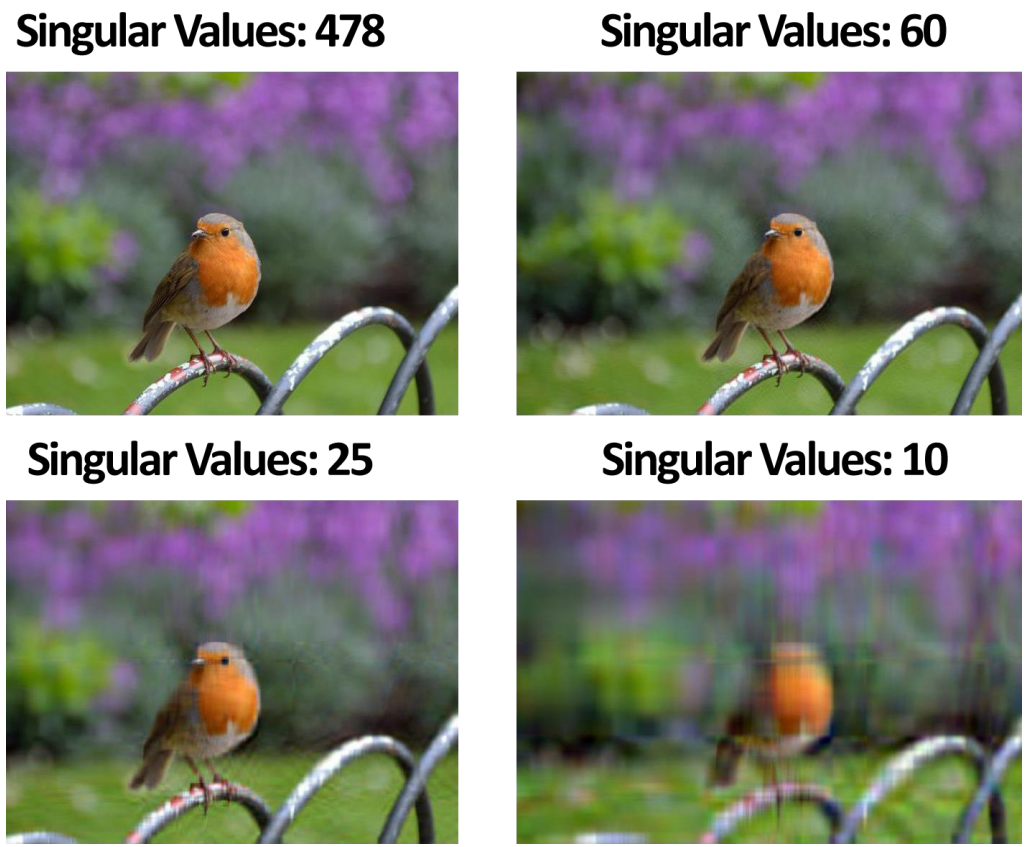
```
from sklearn.decomposition  import TruncatedSVD
import numpy as np

np.random.seed(0)
X = np.random.rand(100,  100)

# four components
svd = TruncatedSVD(n_components=4,  n_iter=10, random_state=5)

U = svd.fit_transform(X)
Sigma = np.diag(svd.singular_values_)
V = svd.components_


# in case we want to do the multiplication

# U x Sigma
U_x_Sigma = np.dot(U, Sigma)

# (U x Sigma) x V
U_Sigma_V = np.dot(U_x_Sigma , V)
```



https://predictivehacks.com/?all-tips=svd-with-scikit-learn

Application of SVD: Compression

- For many images, k much smaller than n can be used to reconstruct the image provided that a very low-resolution version of the image is sufficient.
- Thus, one could use SVD as a compression method.



https://www.cs.cmu.edu/~venkatg/teaching/CStheory-infoage/book-chapter-4.pdf
https://github.com/kounelisagis/Image-Compression---SVD

Summary

- Decision Trees provide interpretable, rule-based models for classification and regression but may overfit due to high variance.
- Ensemble Learning improves performance by combining multiple weak learners into a strong learner.
- Random Forests use bagging and feature randomness to reduce variance and improve robustness.
- Gradient Boosting builds models sequentially, correcting previous errors using gradient-based optimization, achieving high accuracy with proper regularization.

Summary

- Dimensionality Reduction addresses redundancy and noise in high-dimensional data by transforming it into a lower-dimensional space.
- PCA is an unsupervised, linear technique that projects data onto orthogonal components capturing maximum variance.
- PCA supports visualization, compression, and noise reduction, and is sensitive to feature scaling.
- Practical variants include Randomized PCA, Incremental PCA, and Kernel PCA for large-scale and nonlinear data.